数据结构

孙逊　字 憧朋

北京航空航天大学　软件工程硕士

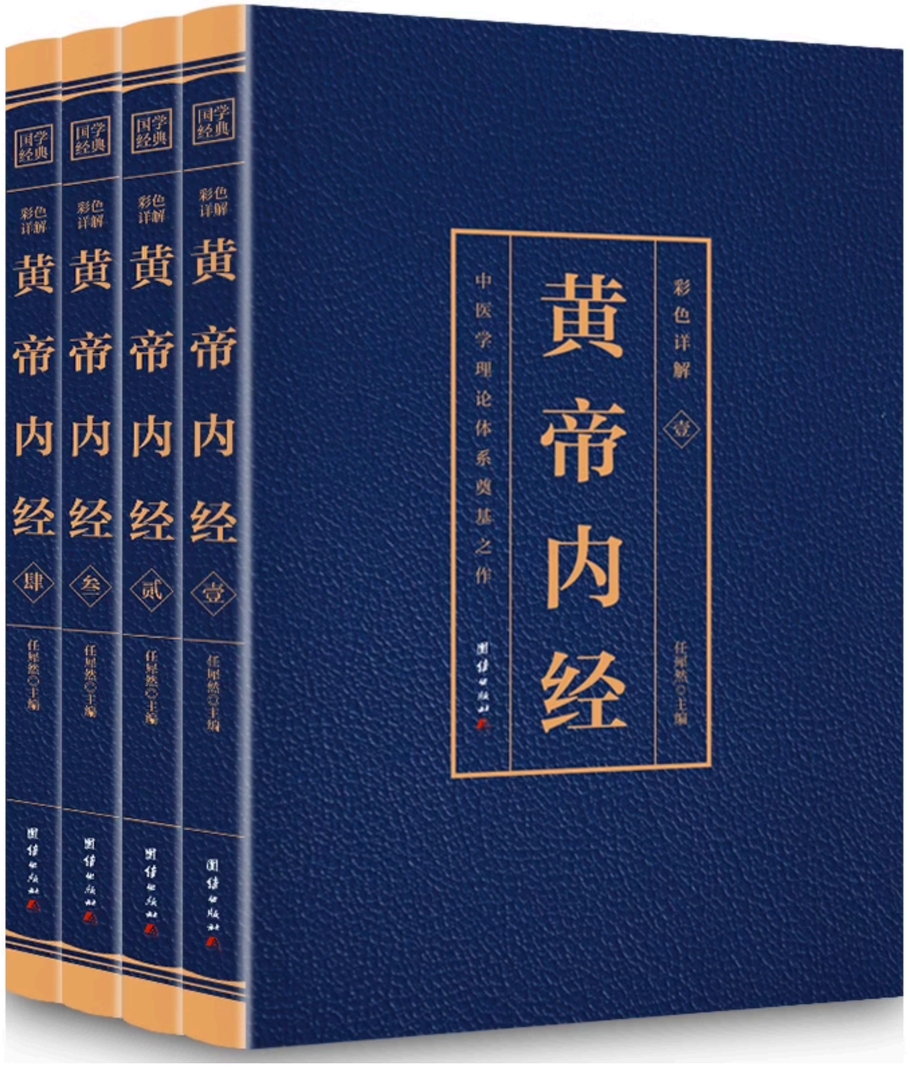
西安交通大学研究生院　特聘专家

百度技术学院　特约讲师

阿里云大学　认证驻场讲师

街舞少年

相声票友



中医学徒

逊哥带你学计算机
UID: 454637713

必修考试课

成绩分配比例

考试成绩 60%

平时成绩 40%

平时成绩奖惩细则

无故旷课	-40
上课穿拖鞋	-40
上课玩手机	-40
早退	-10
迟到	-5
请假	-5
积极回答问题一次	+5

课程目标

理解并掌握课程中所学数据结构与算法

在日常学习及未来工作中可以利用所学知识解决实际问题

本节内容

数据结构课程的学习意义

学习安排

初步认识数据结构

前置知识点复习

数据结构课程的学习意义

启发并扎实对计算机、编程的理解，帮助快速上手其它编程语言

计算机考研必考

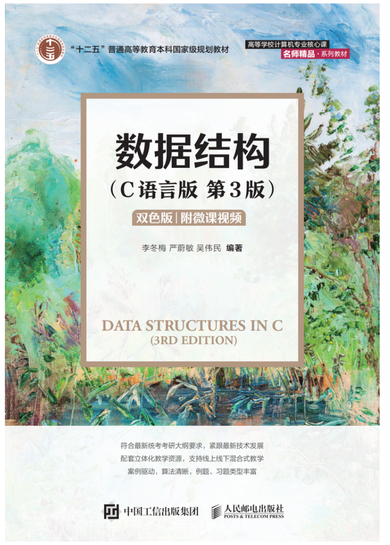
专业必修课（为了学分）

学习安排

课上学习 + 课下练习 + 完成实验

课上学习

老师讲授为主



教材为辅

初步认识数据结构

什么是数据结构?

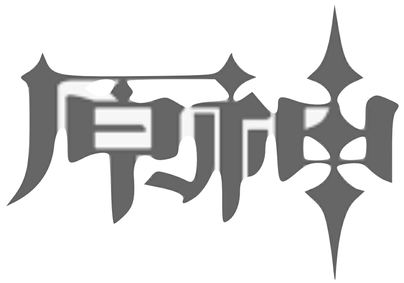
数据 结构

数据 + 结构

什么是数据?

小红书

王者荣耀

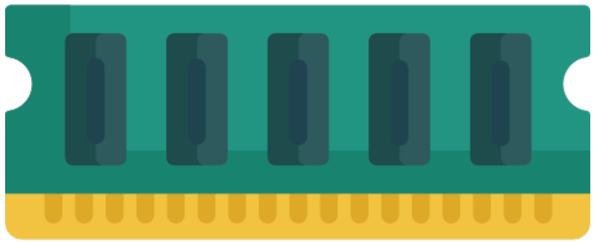

原神

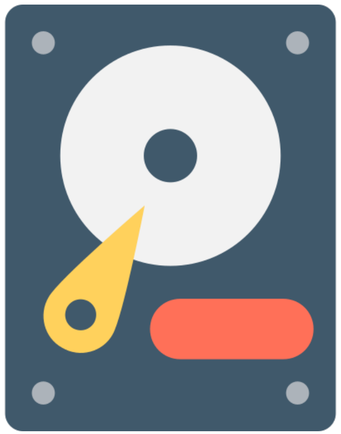
文本

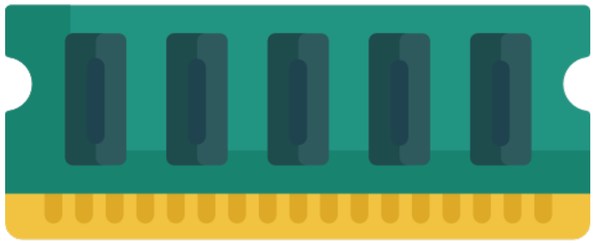
音频

视频

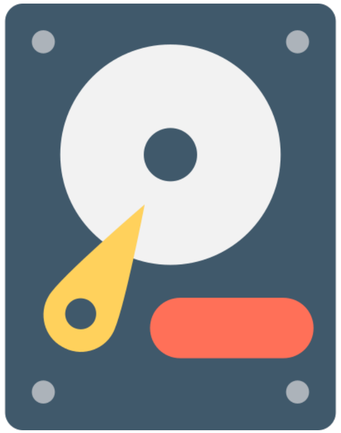
这些数据存在哪?







正在运行中的数据



“持久化”的数据

什么是结构?

孙尚香：女，18岁，擅长打射手位置。

李白：今年30岁，男性，比较喜欢打野。

赵云：20岁，男，和李白一样擅长打野。

吕布：22岁，男，擅长上单战士位置。

安琪拉：19岁，女，中单法师。

姓名	性别	年龄	职业	擅长位置
孙尚香	女	18	射手	下路
李白	男	30	刺客	打野
赵云	男	20	刺客	打野
吕布	男	22	战士	上路
安琪拉	女	19	法师	中路

姓名：孙尚香

性别：女

年龄：18　职业：射手

擅长位置：下路

姓名：李白

性别：男

年龄：30　职业：刺客

擅长位置：打野

姓名：赵云

性别：男

年龄：20　职业：刺客

擅长位置：打野

姓名：吕布

性别：男

年龄：22　职业：战士

擅长位置：上路

姓名：安琪拉

性别：女

年龄：19　职业：法师

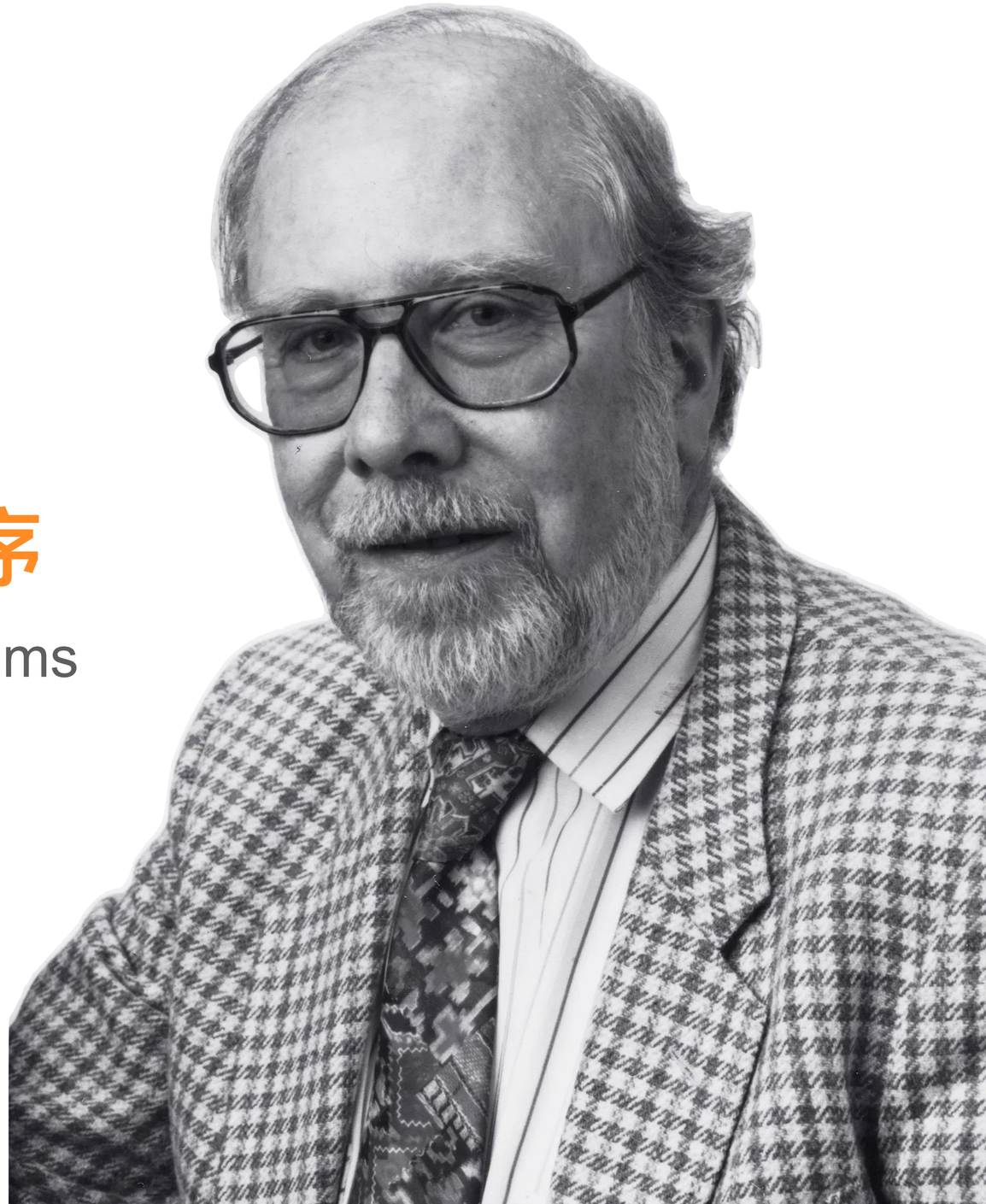
擅长位置：中路

在一些特定的、特殊的需求场景下，我们以往所学的数据类型，无法基于需求合理地组织数据，此时就需要我们自己设计一套新的数据组织形式来解决问题，也就是数据结构。

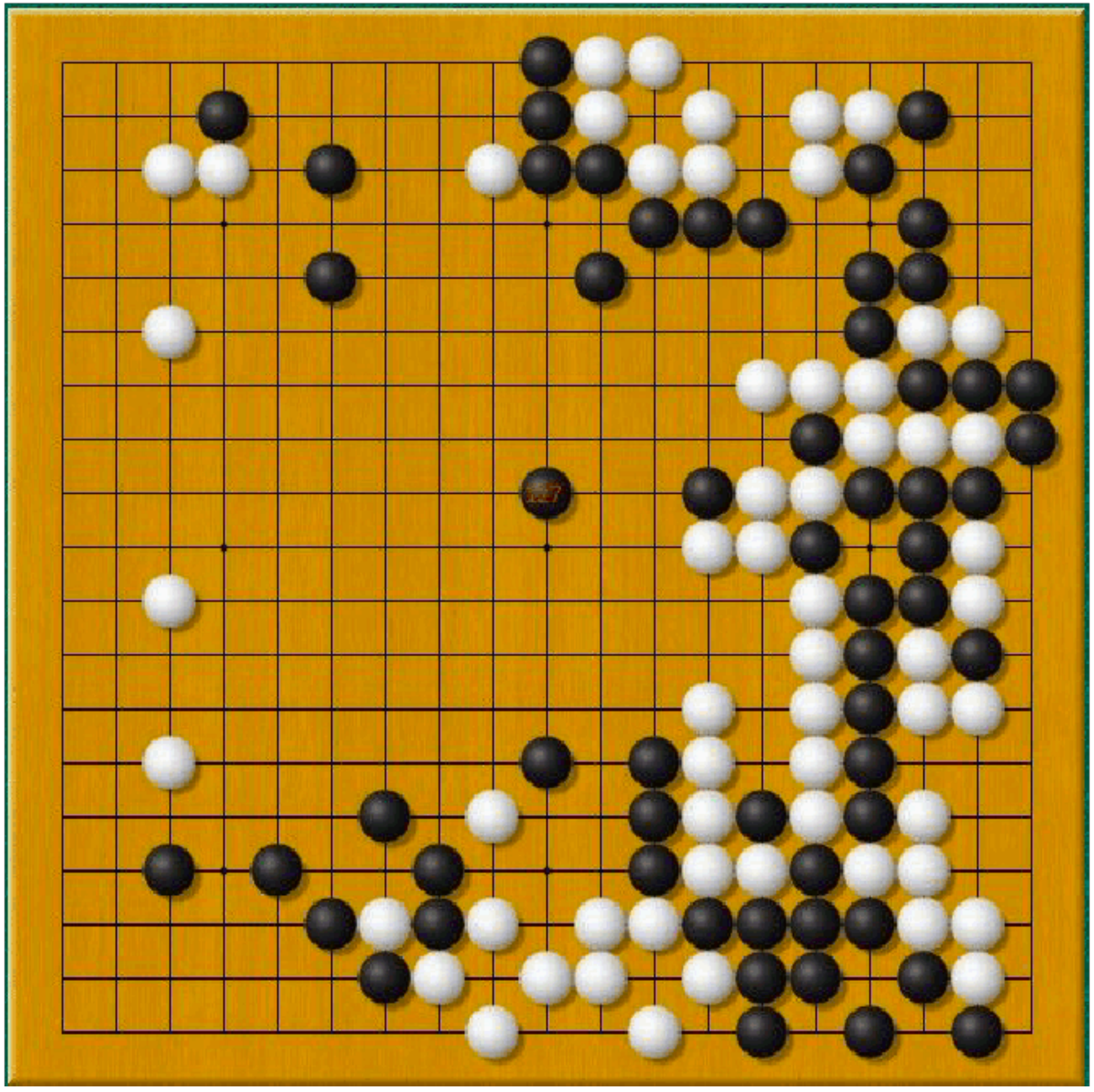
数据结构是一种存储、组织数据的方式，旨在便于访问和修改。没有一种单一的数据结构对所有用途均有效，所以需要知道几种数据结构优势和局限。

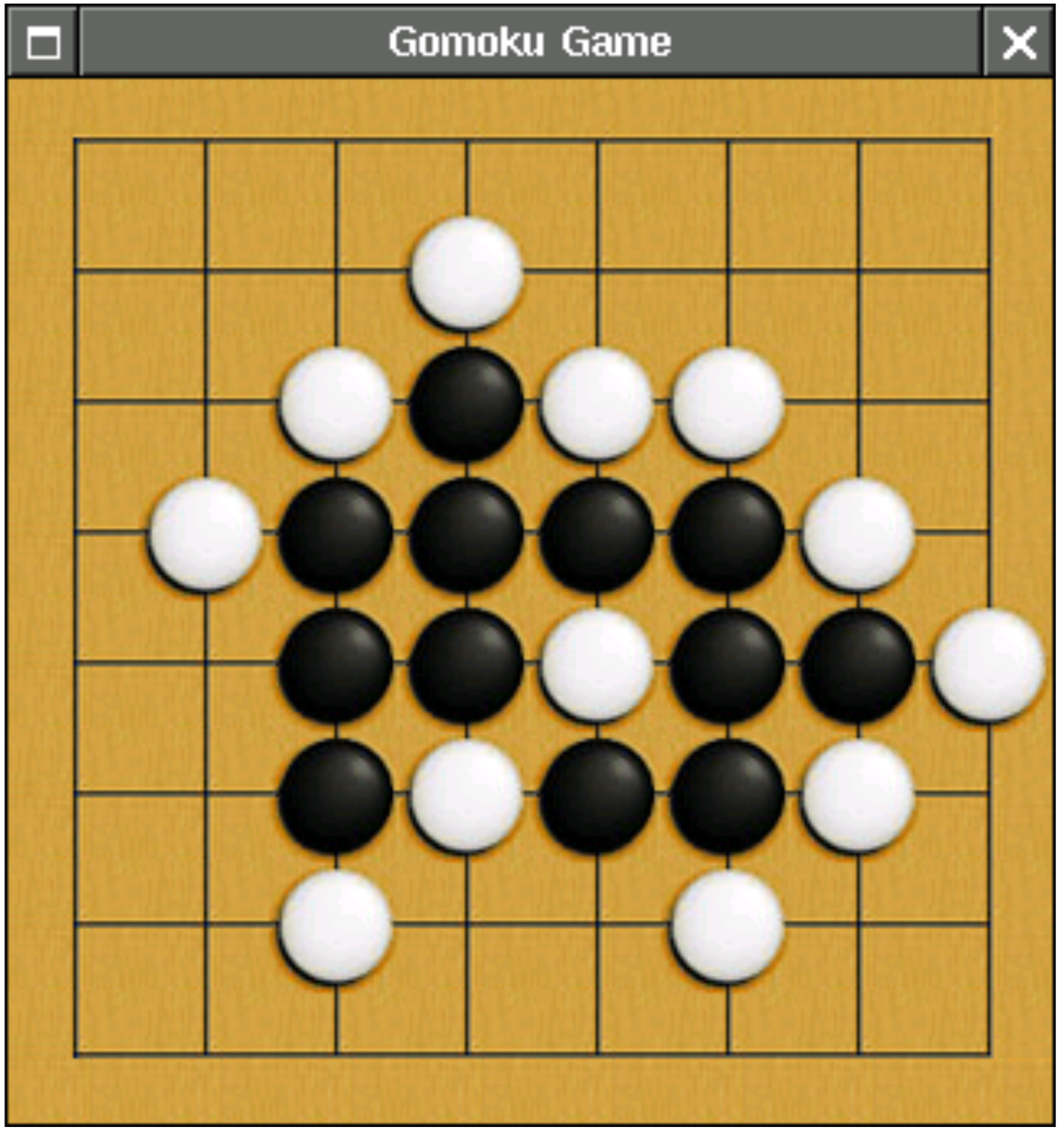
算法 + **数据结构** = **程序**

Algorithms + Data Structures = Programs

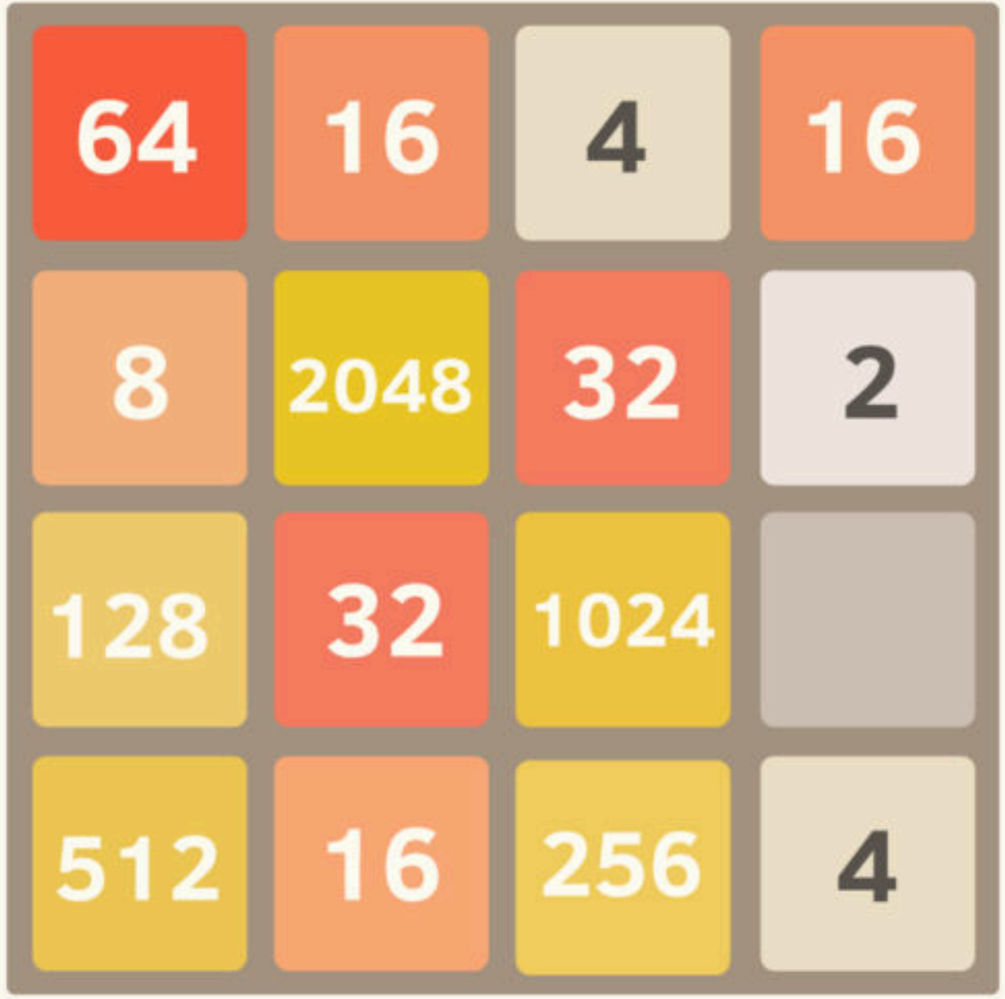


算法是一个有穷规则的集合，它用规则规定了解决某一特定类型问题的运算序列，或者规定了任务执行或问题求解的一系列步骤。

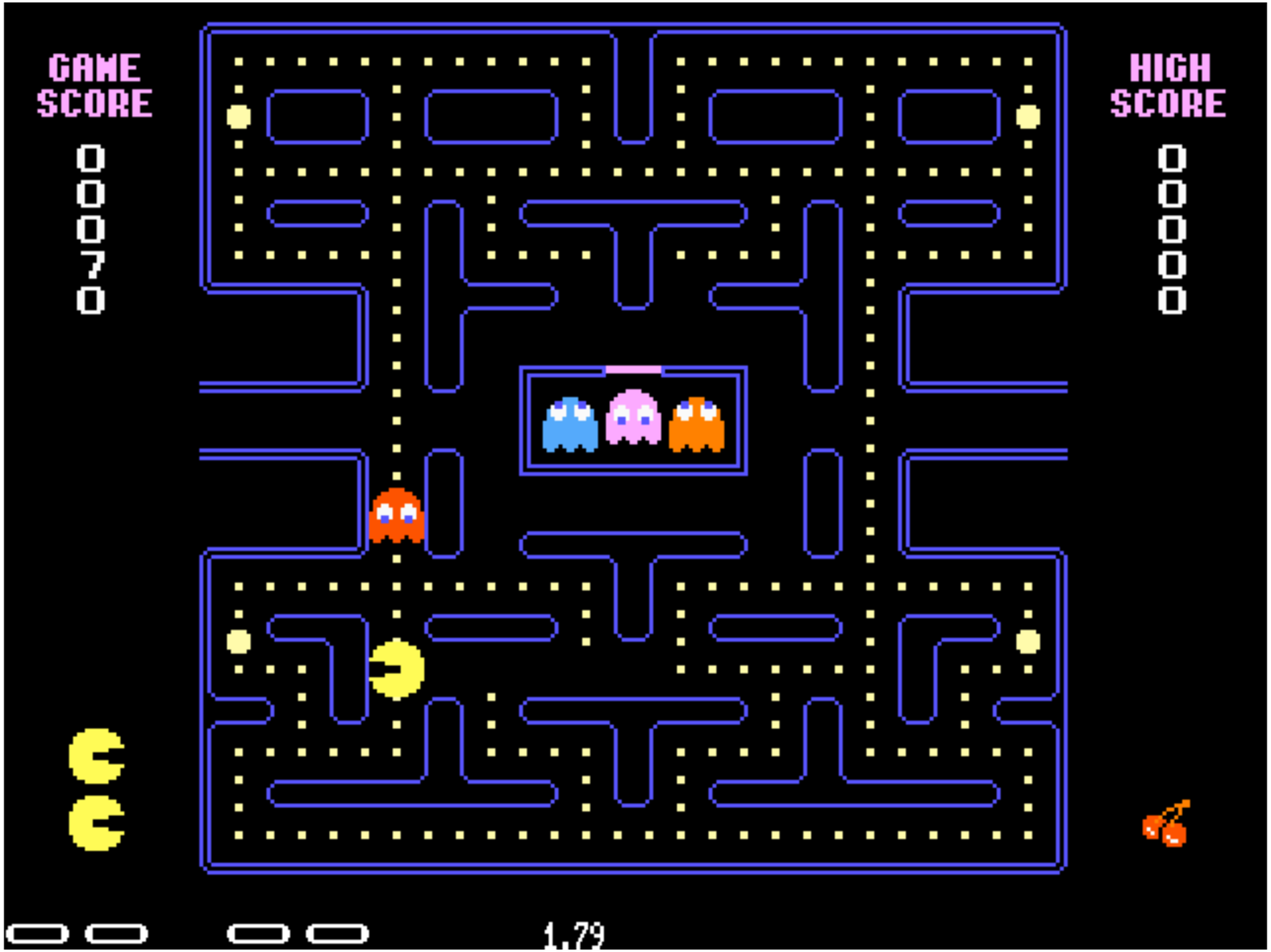



Gomoku Game

Reach the 2048 tile to win the game!



PLAY


GAME
SCORE
00070
HIGH
SCORE
00000
1.79

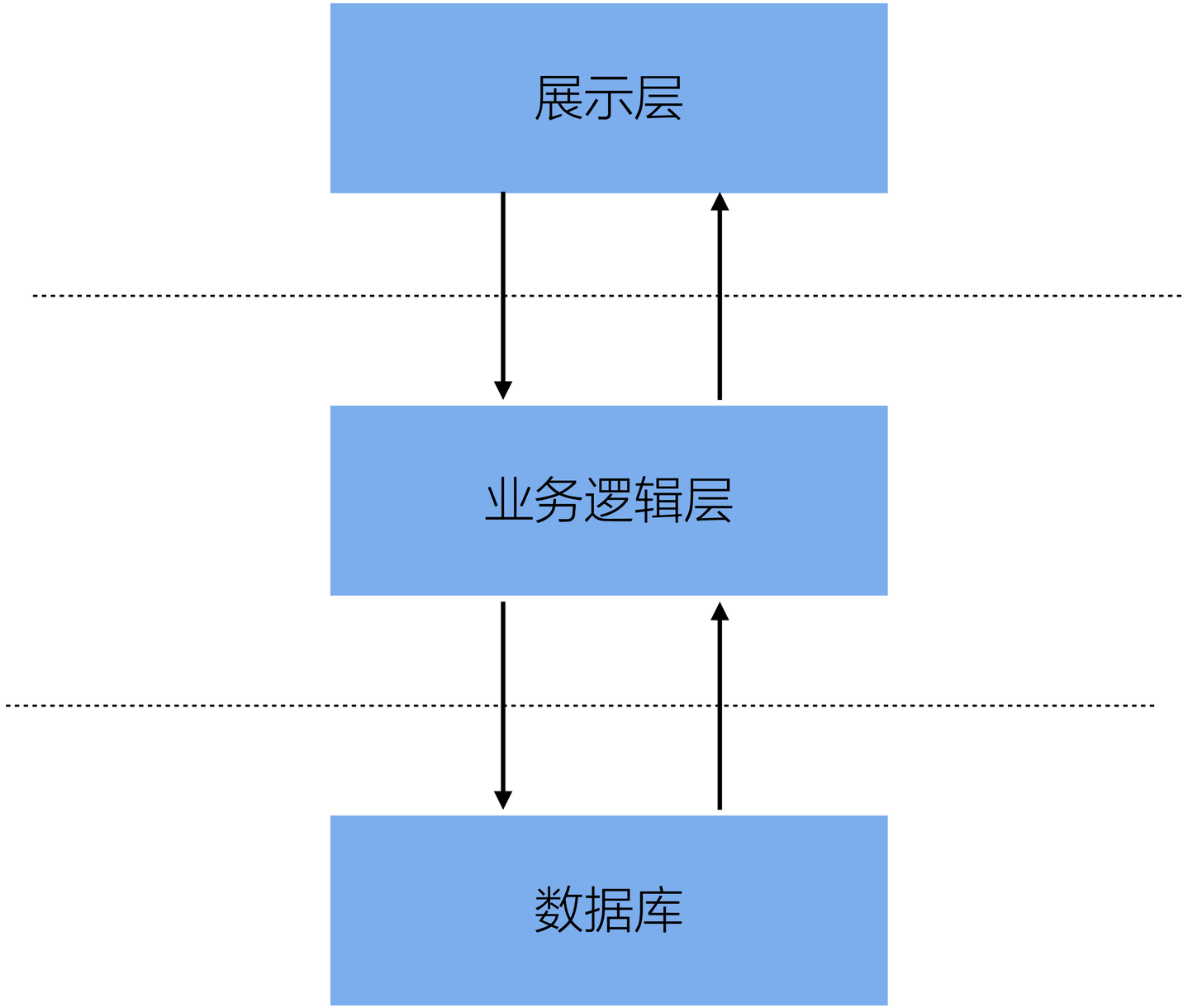
位置信息

宝安中心A地铁口

宝安中心A地铁口

数据结构中的“数据”是指哪里的数据?

数据结构中的讨论数据，大多数情况下是指计算机程序运行过程中临时存储在内存中的数据


展示层
业务逻辑层
数据库

这门课学什么？

课程学习内容

数据结构

线性表、栈、队列、数组、树与二叉树、图

查找、排序

基于 C 语言描述的数据结构需要哪些前置知识?

函数

数组

字符串

指针

内存解析

结构体

函数

可以实现某个具体功能的代码块

增加代码的复用性

降低编程难度

函数不被调用是不会执行的

对内隐藏细节，对外暴露接口

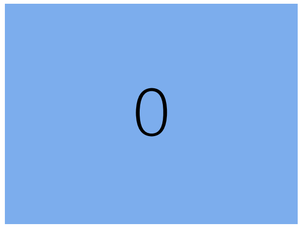
```
<返回值类型|void> 函数名([参数列表])
{
    //函数体

    [return 返回值]
}
```

案例：函数/test1.c　函数/test2.c

sum

0

sum



函数的四种形式

无参数，无返回值　　案例：函数/test3.c

无参数，有返回值　　案例：函数/test4.c

有参数，无返回值　　案例：函数/test5.c

有参数，有返回值　　案例：函数/test6.c

日常写代码，函数什么时候加参数，什么时候加返回值?

根据需求编写！！！

字符串

字符串初始化

```
char str[] = "HelloWorld";
```

H	e	l	l	o	W	o	r	l	d	\0

```
char str2[11];
str2 = "HelloWorld";
```

?

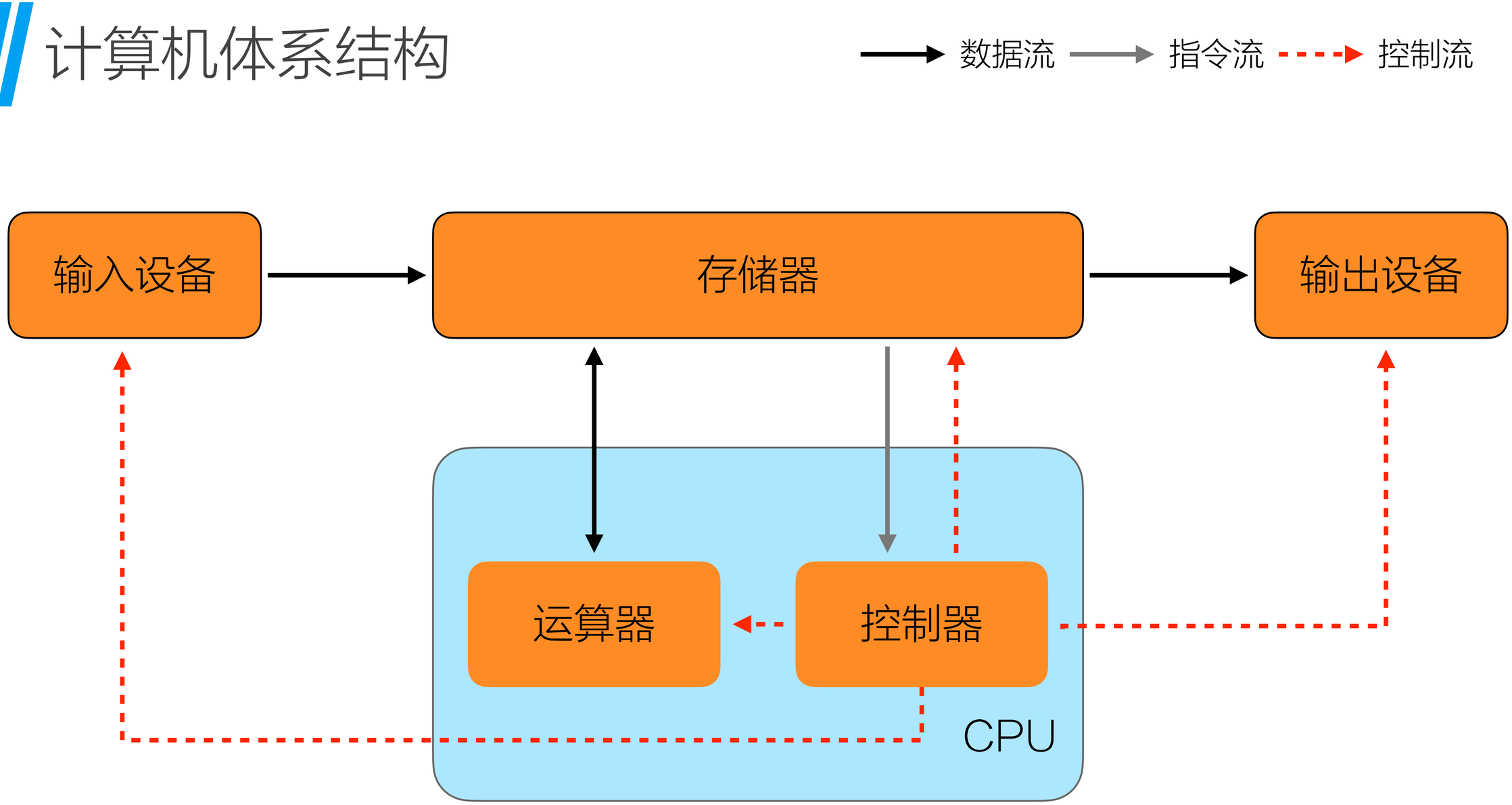
案例：字符串/test1.c

字符串初始化

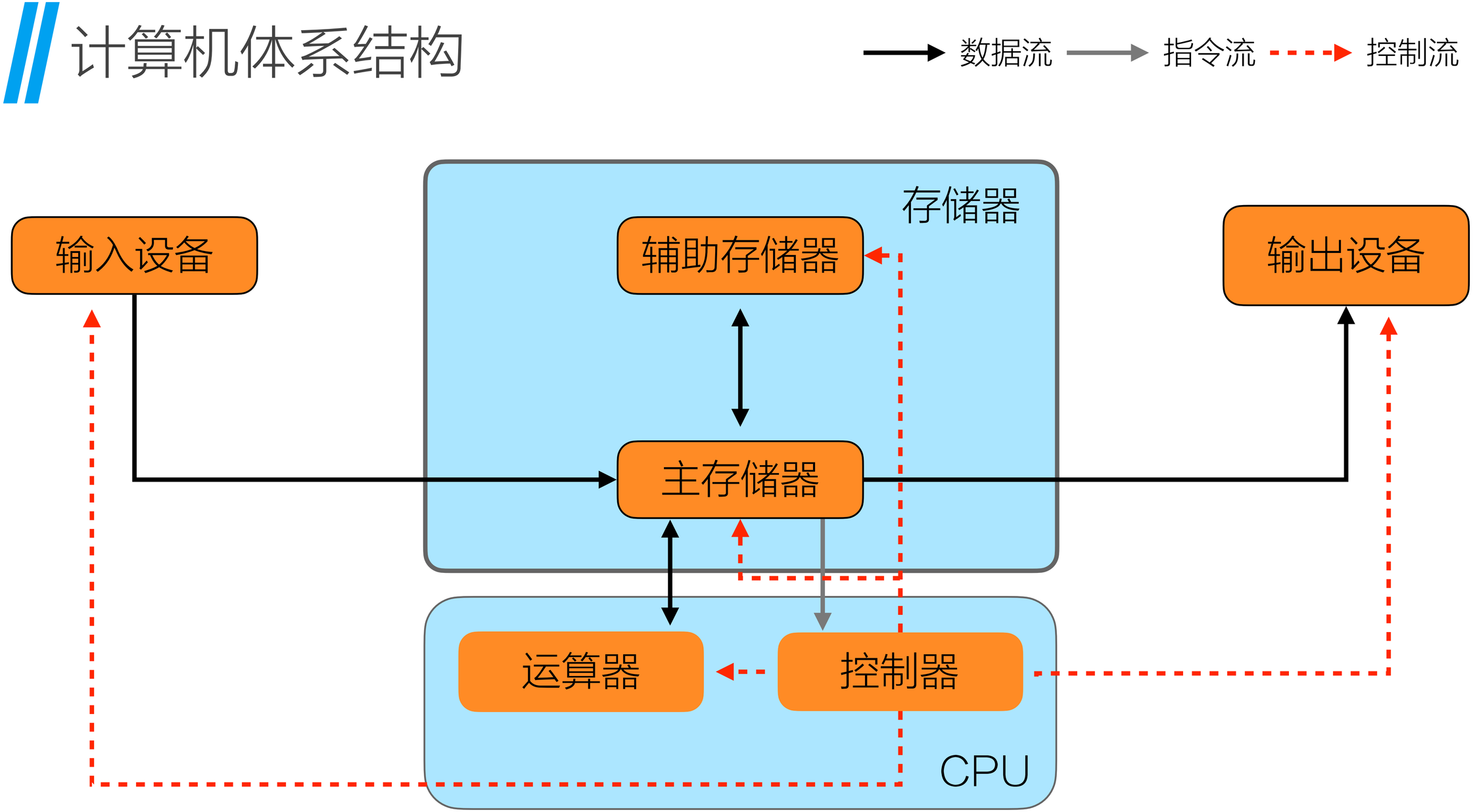
```
#include<stdio.h>
#include<string.h>
int main()
{
    char str2[11];
    strcpy(str2, "HelloWorld");
}
```

案例：字符串/test1.c

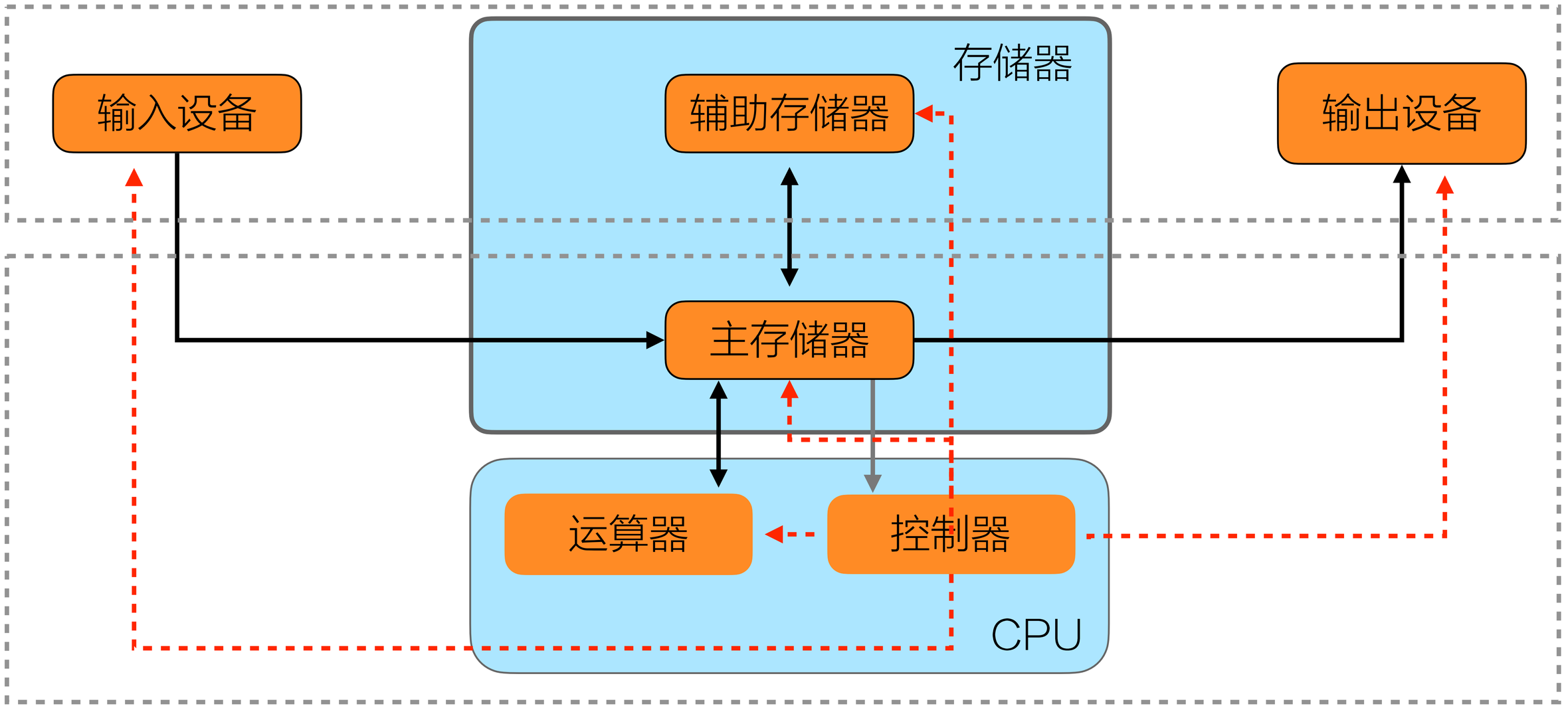
计算机体系结构



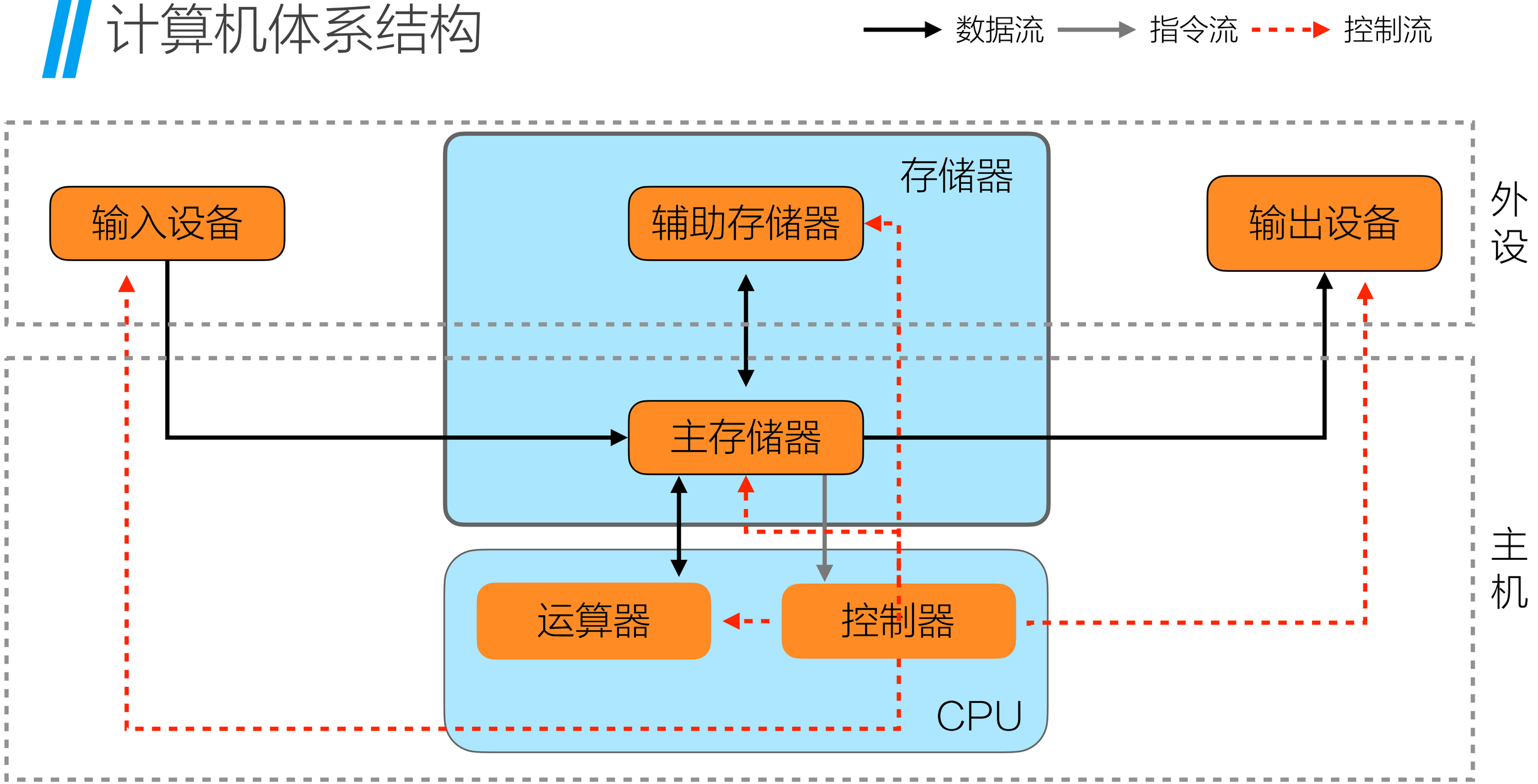
计算机体系结构



计算机体系结构



计算机体系结构

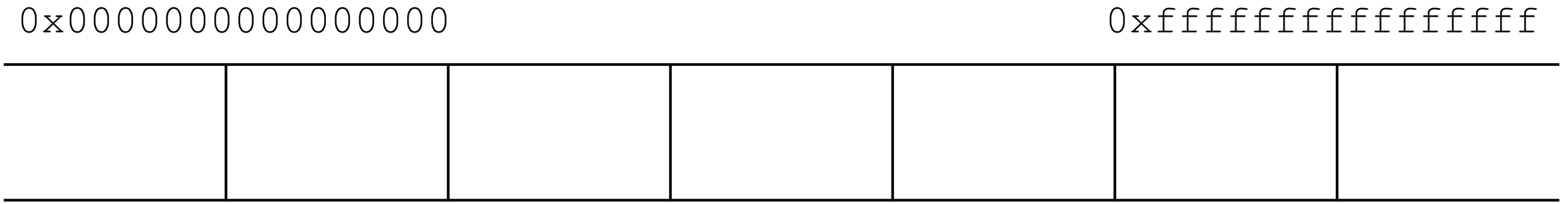


虚拟内存地址

内存条、显卡、各种适配卡都有其各自的存储地址空间。

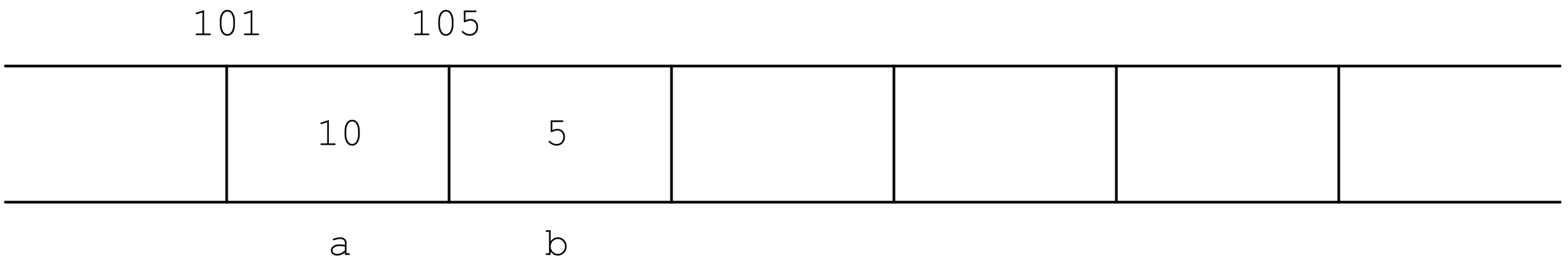
操作系统将这些设备的存储地址空间抽象成一个巨大的一维数组空间。

对于内存的每一个字节会分配一个 32 位或 64 位的编号，这个编号称为内存地址。



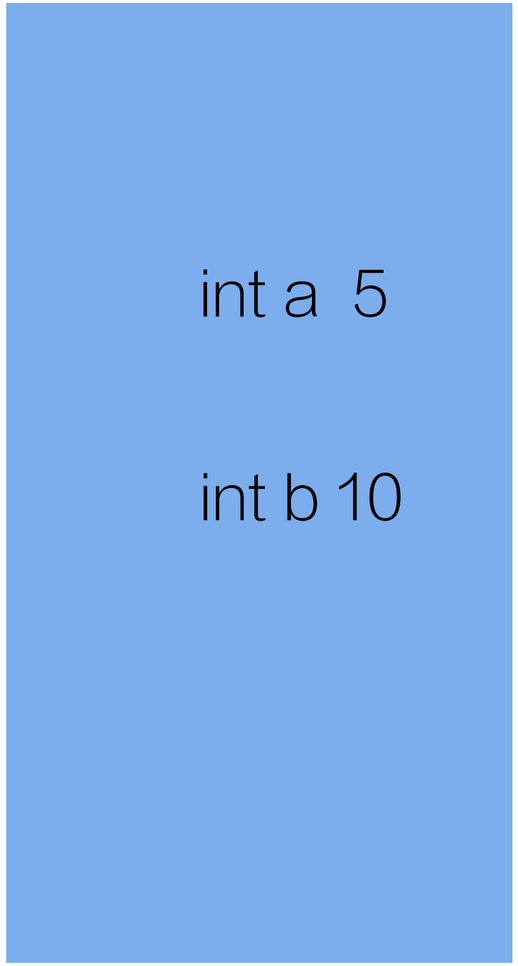
虚拟内存地址

```
int main()
{
    int a = 10;
    int b = 5;
    return 1;
}
```



```
int a = 5;
int b = 10;
```

栈内存



数组

相同数据类型的集合。

数组的长度一旦定义就不能改变。

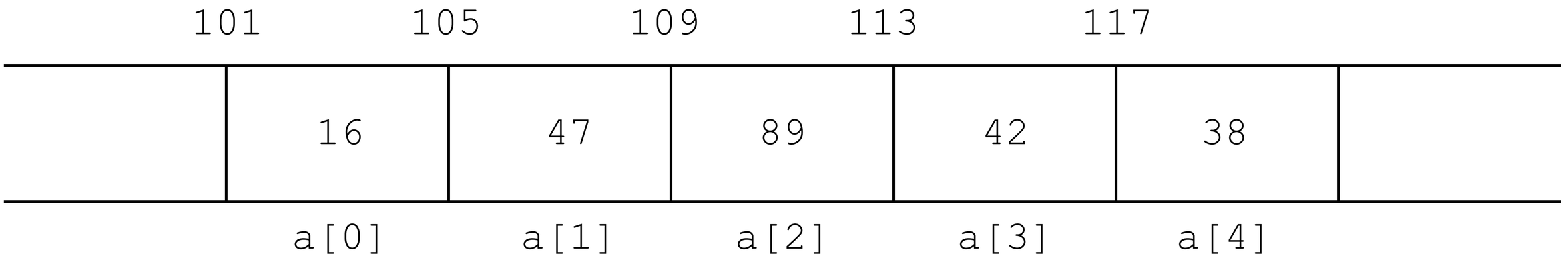
数组中的每一个元素可以用下标表示位置，如果一个为数组中有 n 个元素，那么下标的取值范围是 0~n-1。

数组

```
int main()
{
    int a[] = {16,47,89,42,38};
    for (int i = 0; i < 5; i++)
    {
        printf("%d\n", a[i]);
    }
}
```

案例：数组/test1.c

遍历



获取数组地址

使用取地址符 & 获取数组的地址时，返回的是数组第 0 个元素的内存地址

```
int main()
{
    int a[] = {16,47,89,42,38};
    printf("%p\n", &a);
    printf("%p\n", &a[0]);
}
```

案例：数组/test2.c

数组与 sizeof

```
int main()
{
    int a[] = {16,47,89,42,38};
    printf("%zu\n", sizeof(a));
    printf("%zu\n", sizeof(a[0]));
    int len = sizeof(a) / sizeof(a[0]);
    printf("数组长度为%d\n", len);
}
```

案例：数组/test3.c

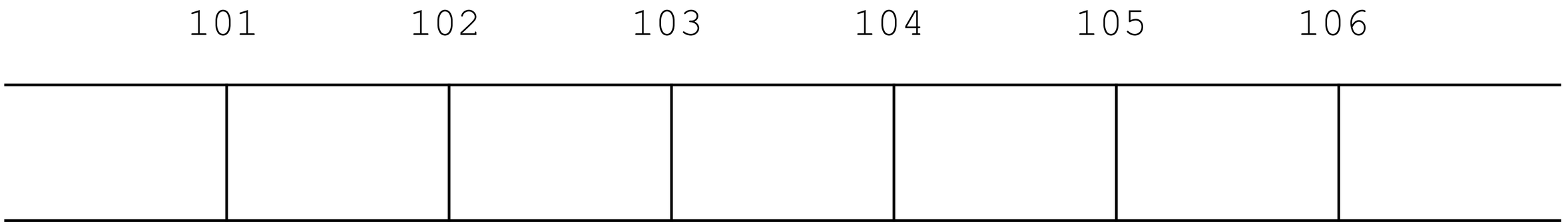
指针

C 程序员新手和老手的其中之一差别在于**是否对指针有深刻的理解、高效的使用指针**。

理解指针的关键在于**理解 C 程序如何管理内存**。

指针的定义

指针是用来存放内存地址的变量。



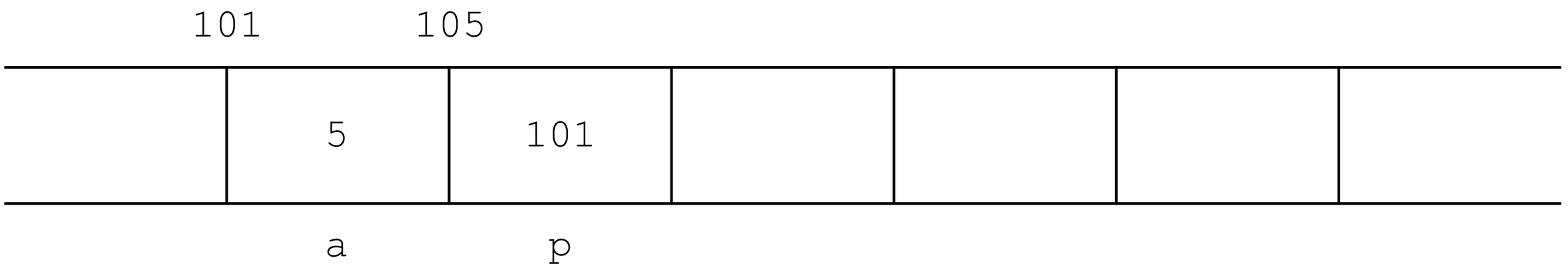
指针的声明

```
int a;    //声明一个整型变量
int *p;   //声明一个指针变量，该指针指向一个int类型值的内存地址

//星号 * 两边的空格无关紧要，下面的声明是等价的
int* p;
int *p;
int * p;
int*p;
```

指针的使用

```
int a;
int *p;
a = 5;
p = &a;
```

案例：指针/test1.c



间接引用操作符 *

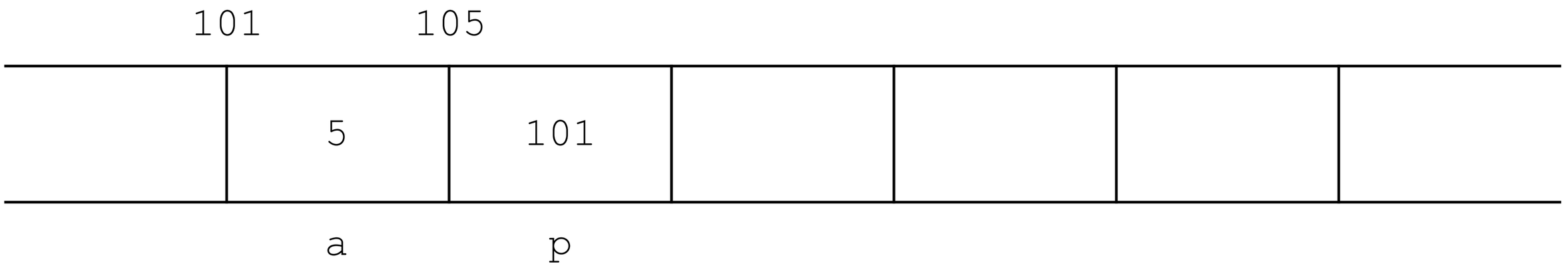
间接引用操作符 * 返回指针变量指向地址的值

通常把这个操作叫做“解引用指针”

英文叫做 dereferencing a pointer

```
int a = 5;
int *p = &a;
printf("%d\n", *p);
*p = 100;
printf("%d\n", a)
```

案例：指针/test2.c



间接引用操作符 *

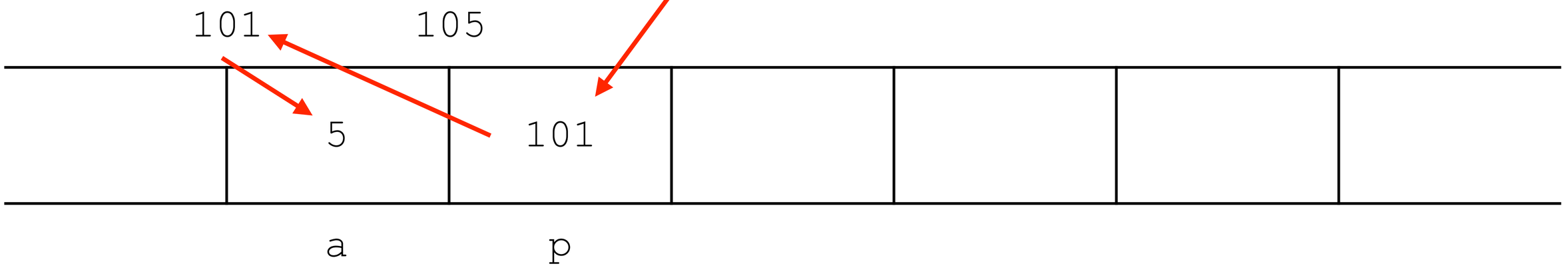
间接引用操作符 * 返回指针变量的指向地址的值

通常把这个操作叫做“解引用指针”

英文叫做 dereferencing a pointer

```
int a = 5;
int *p = &a;
printf("%d\n", *p);
*p = 100;
printf("%d\n", a)
```

案例：指针/test2.c



间接引用操作符 *

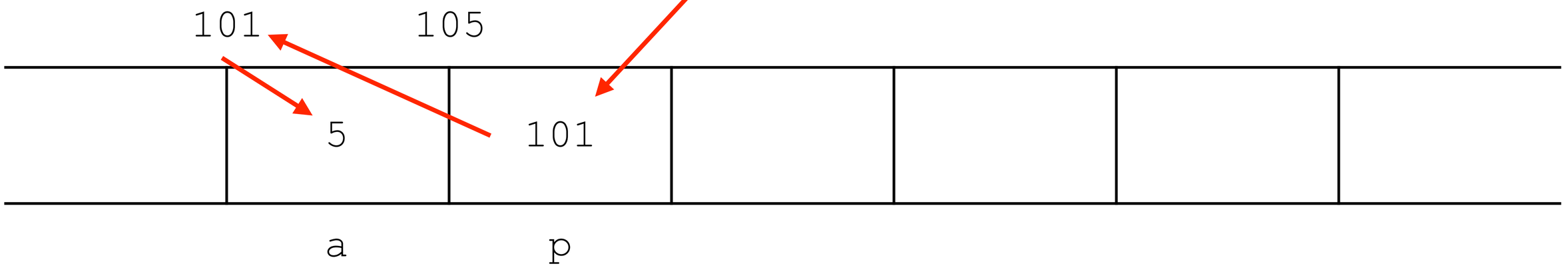
间接引用操作符 * 返回指针变量的指向地址的值

通常把这个操作叫做“解引用指针”

英文叫做 dereferencing a pointer

```
int a = 5;
int *p = &a;
printf("%d\n", *p);
*p = 100;
printf("%d\n", a)
```

案例：指针/test2.c



间接引用操作符 *

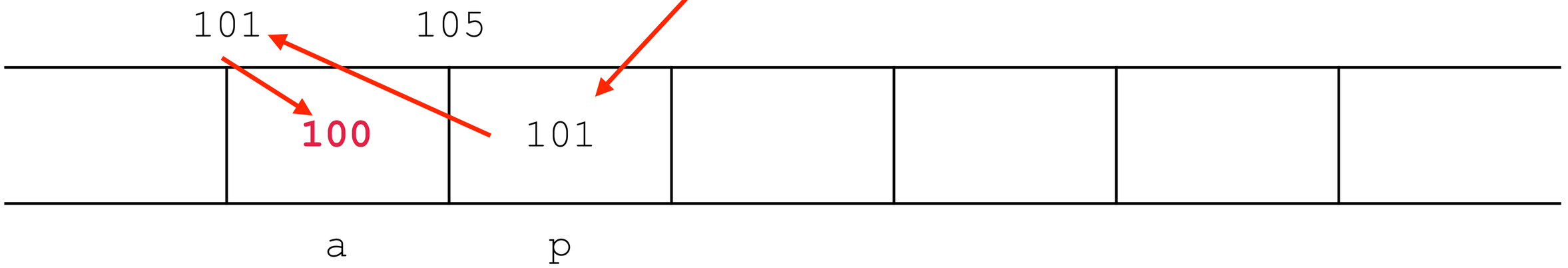
间接引用操作符 * 返回指针变量的指向地址的值

通常把这个操作叫做“解引用指针”

英文叫做 dereferencing a pointer

```
int a = 5;
int *p = &a;
printf("%d\n", *p);
*p = 100;
printf("%d\n", a)
```

案例：指针/test2.c



间接引用操作符 *

间接引用操作符 * 返回指针变量的指向地址的值

通常把这个操作叫做“解引用指针”

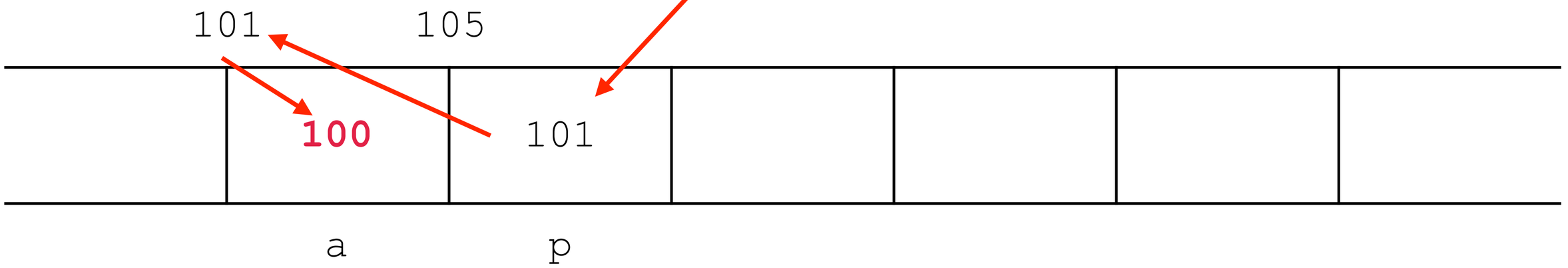
英文叫做 dereferencing a pointer

```
int a = 5;
int *p = &a;
printf("%d\n", *p);
*p = 100;
printf("%d\n", a)
```

从指定的地址读取

存储到指定的地址

案例：指针/test2.c



指针与数组

在 C 语言中，指针与数组的关系十分密切。

通过数组下标能完成的操作都可以通过指针完成。

一般来说，用指针编写的程序比用数组下标编写的程序执行速度快。

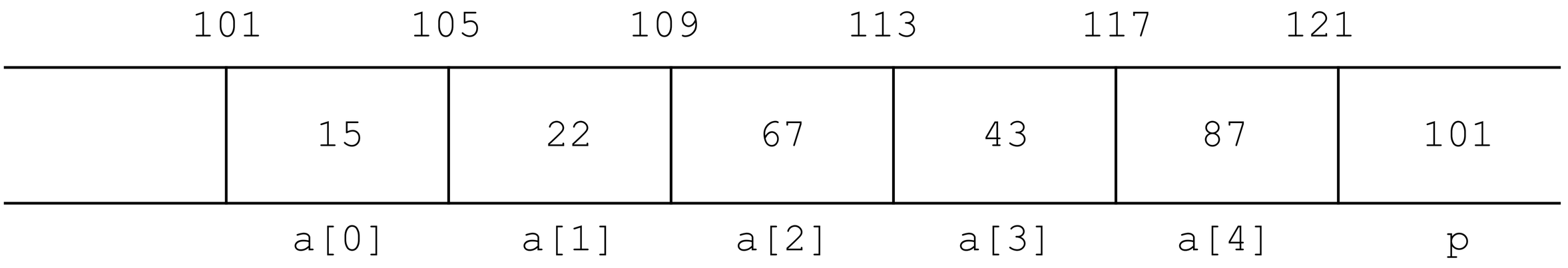
```c
int a[] = {15,22,67,43,87};
int *p;
p = a;

printf("%p\n", a);
printf("%p\n", p);
printf("%d\n", *p);
```

案例：指针/test5.c

指针与数组

```c
int a[] = {15,22,67,43,87};
int *p;
p = a;
for (int i = 0; i < sizeof(a)/sizeof(a[0]); i++)
{
    printf("%d\n", a[i]);
}

for (int i = 0; i < sizeof(a)/sizeof(a[0]); i++)
{
    printf("%d\n", *(p + i));
}
```

案例：指针/test6.c



指针做算术运算

给指针加上一个整数，实际上加的是这个整数和指针数据类型对应字节数的乘积。

```c
int a = 5;
int *p = &a;
printf("%p\n", p);
p++;
printf("%p\n", p);
return 0;
```

案例：指针/test7.c

p++ ⟶ p = p + 1 ⟶ p = p + 1 × 4

各数据类型字节数

有符号数	无符号数	32位	64位
char	unsigned char	1	1
short	unsigned short	2	2
int	unsigned int	4	4
long	unsigned long	4	8
float		4	8
double		8	8

指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```c
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("a=%d, b=%d\n", a,b);
}

int main(int argc, char const *argv[])
{
    int m = 5;
    int n = 10;
    swap(m, n);
    printf("m=%d, n=%d\n", m, n);
    return 0;
}
```

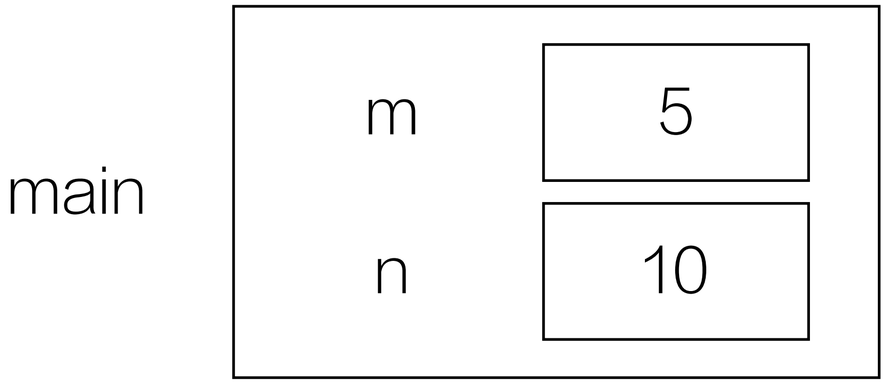
案例：指针/test3.c

指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```c
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("a=%d, b=%d\n", a,b);
}

int main(int argc, char const *argv[])
{
    int m = 5;
    int n = 10;
    swap(m, n);
    printf("m=%d, n=%d\n", m, n);
    return 0;
}
```

案例：指针/test3.c

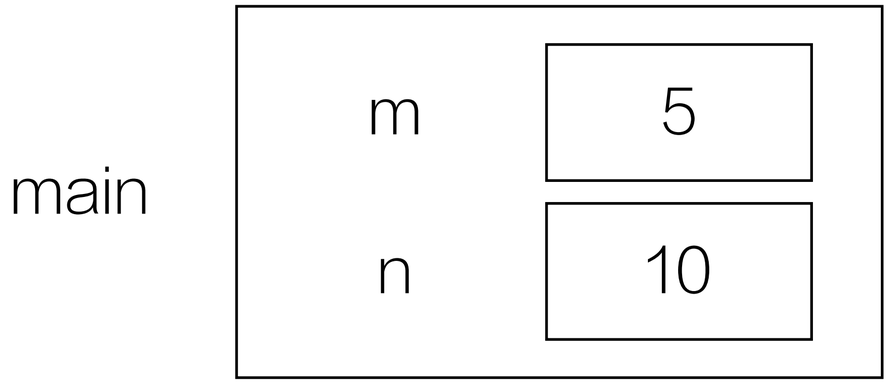


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```c
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("a=%d, b=%d\n", a,b);
}

int main(int argc, char const *argv[])
{
    int m = 5;
    int n = 10;
    swap(m, n);
    printf("m=%d, n=%d\n", m, n);
    return 0;
}
```

案例：指针/test3.c

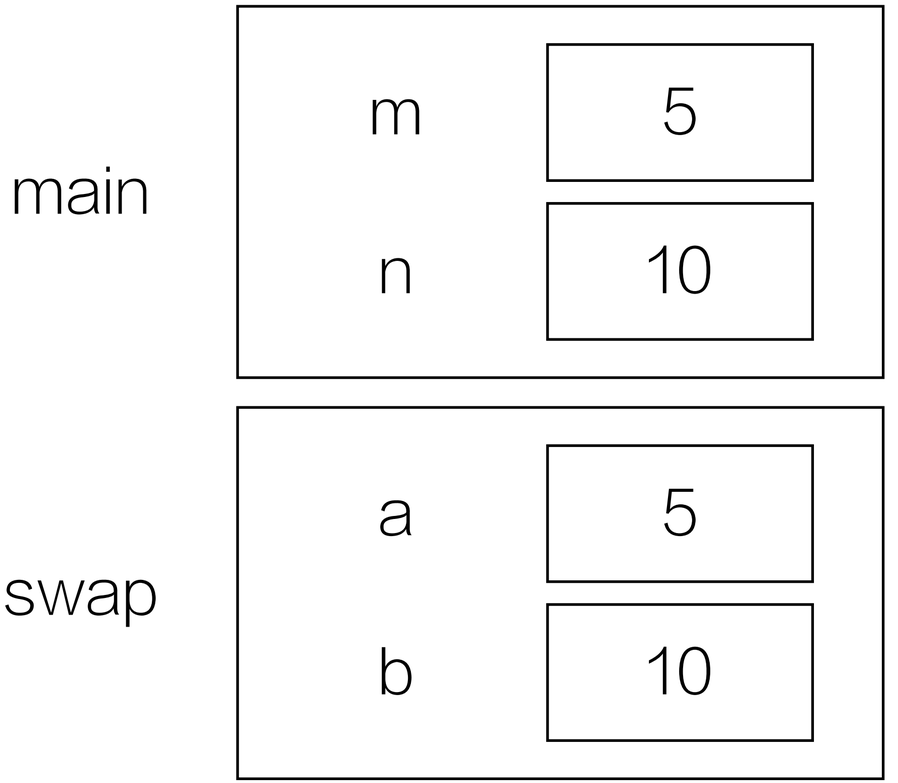


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```c
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("a=%d, b=%d\n", a,b);
}

int main(int argc, char const *argv[])
{
    int m = 5;
    int n = 10;
    swap(m, n);
    printf("m=%d, n=%d\n", m, n);
    return 0;
}
```

案例：指针/test3.c

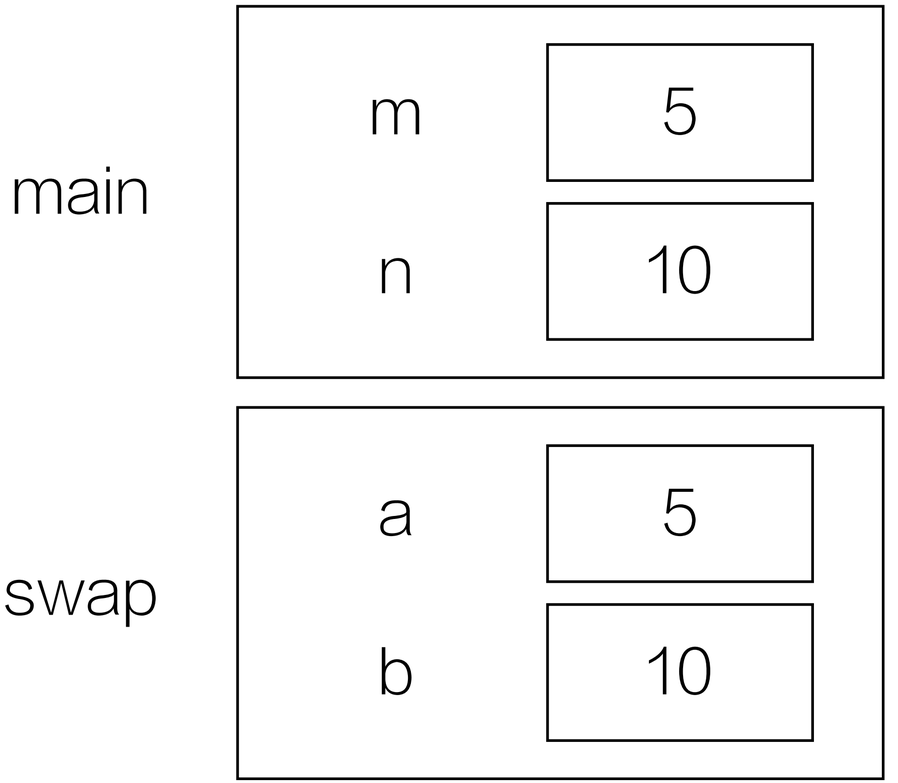


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```c
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("a=%d, b=%d\n", a,b);
}

int main(int argc, char const *argv[])
{
    int m = 5;
    int n = 10;
    swap(m, n);
    printf("m=%d, n=%d\n", m, n);
    return 0;
}
```

案例：指针/test3.c

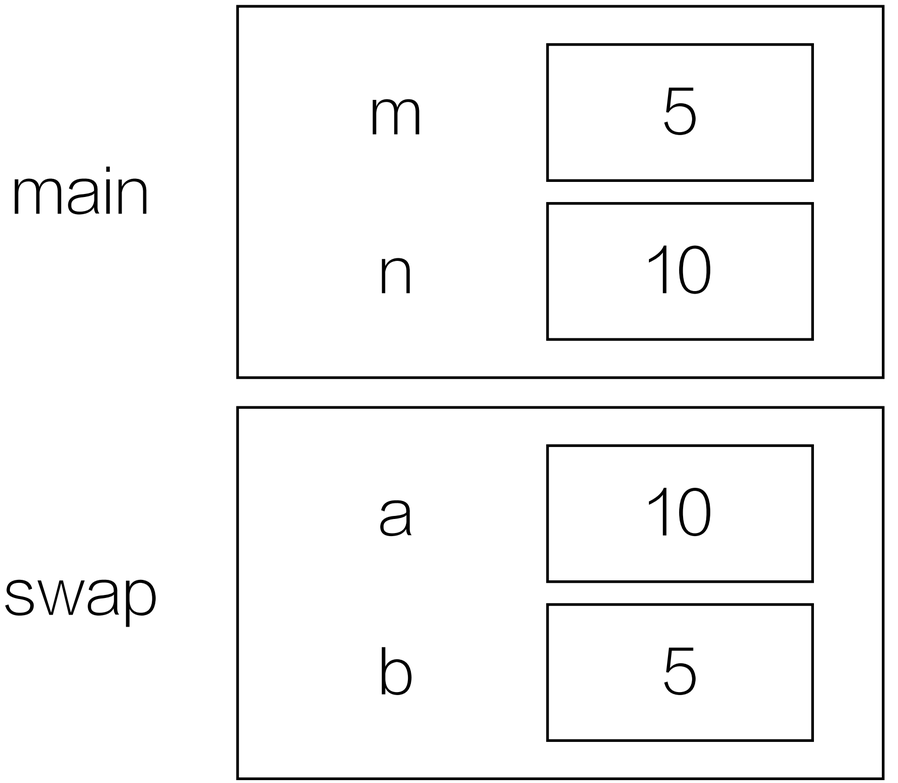


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```c
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
    printf("a=%d, b=%d\n", a,b);
}

int main(int argc, char const *argv[])
{
    int m = 5;
    int n = 10;
    swap(m, n);
    printf("m=%d, n=%d\n", m, n);
    return 0;
}
```

案例：指针/test3.c



指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```
 3  void swap(int *a, int *b)
 4  {
 5      int temp;
 6      temp = *a;
 7      *a = *b;
 8      *b = temp;
 9      printf("a=%d, b=%d\n", *a, *b);
10  }
11
12  int main(int argc, char const *argv[])
13  {
14      int m = 5;
15      int n = 10;
16      swap(&m, &n);
17      printf("m=%d, n=%d\n", m, n);
18      return 0;
19  }
```

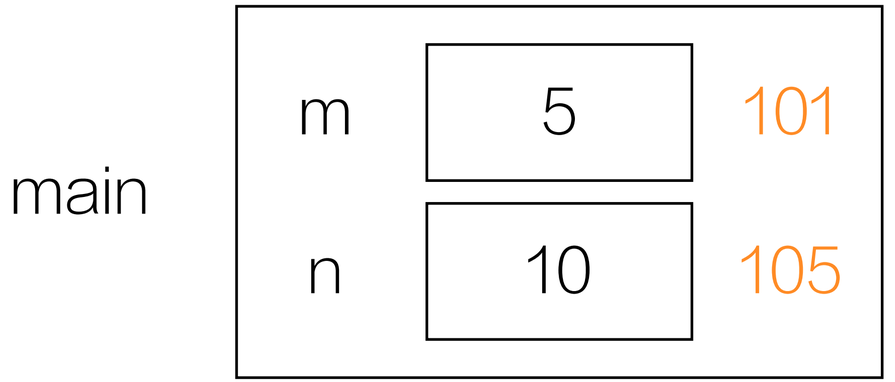
案例：指针/test4.c

指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```
3  void swap(int *a, int *b)
4  {
5      int temp;
6      temp = *a;
7      *a = *b;
8      *b = temp;
9      printf("a=%d, b=%d\n", *a, *b);
10 }
11
12 int main(int argc, char const *argv[])
13 {
14     int m = 5;
15     int n = 10;
16     swap(&m, &n);
17     printf("m=%d, n=%d\n", m, n);
18     return 0;
19 }
```

案例：指针/test4.c

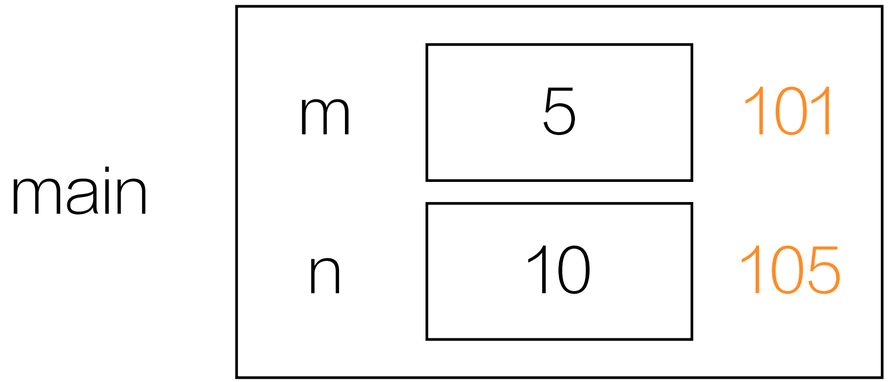


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```
3  void swap(int *a, int *b)
4  {
5      int temp;
6      temp = *a;
7      *a = *b;
8      *b = temp;
9      printf("a=%d, b=%d\n", *a, *b);
10 }
11
12 int main(int argc, char const *argv[])
13 {
14     int m = 5;
15     int n = 10;
16     swap(&m, &n);
17     printf("m=%d, n=%d\n", m, n);
18     return 0;
19 }
```

案例：指针/test4.c

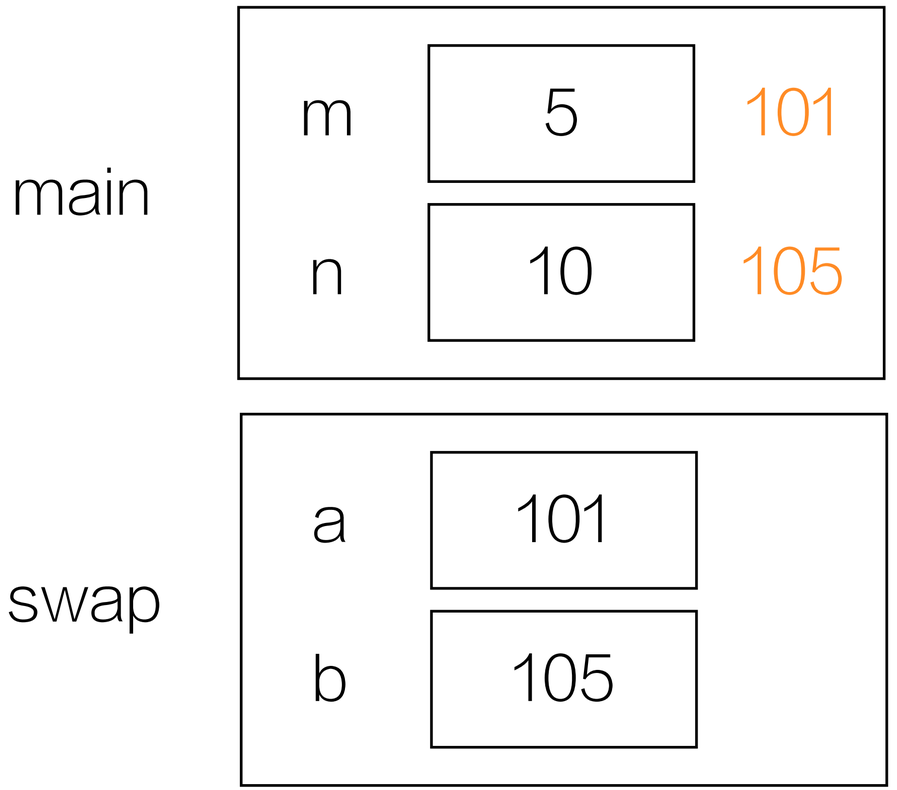


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```
3  void swap(int *a, int *b)
4  {
5      int temp;
6      temp = *a;
7      *a = *b;
8      *b = temp;
9      printf("a=%d, b=%d\n", *a, *b);
10 }
11
12 int main(int argc, char const *argv[])
13 {
14     int m = 5;
15     int n = 10;
16     swap(&m, &n);
17     printf("m=%d, n=%d\n", m, n);
18     return 0;
19 }
```

案例：指针/test4.c

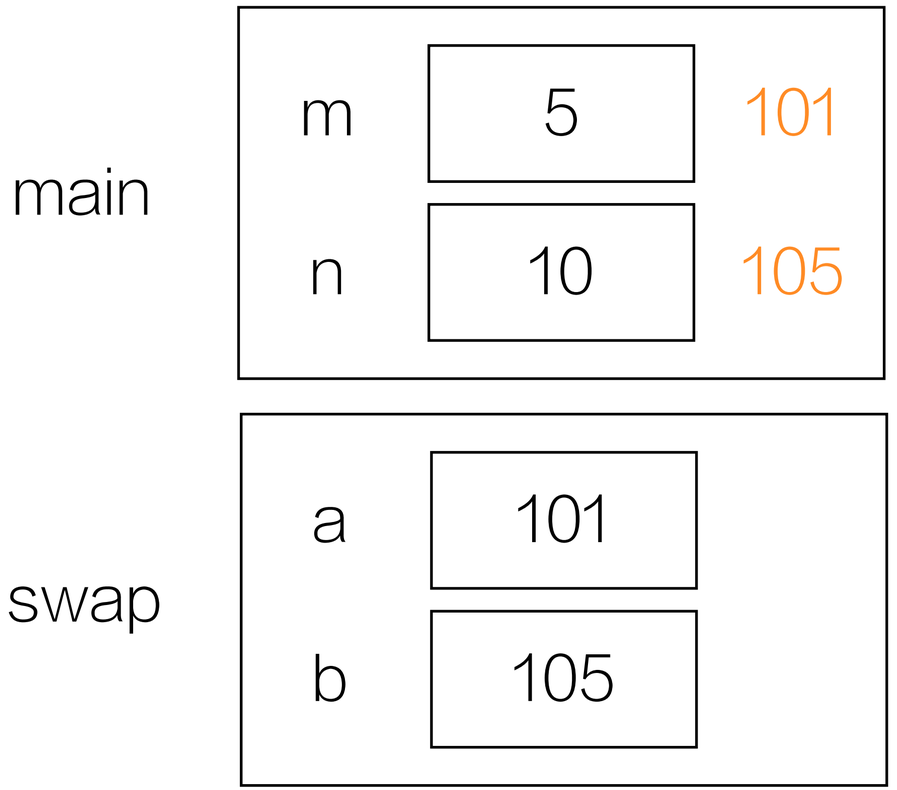


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```
3  void swap(int *a, int *b)
4  {
5      int temp;
6      temp = *a;
7      *a = *b;
8      *b = temp;
9      printf("a=%d, b=%d\n", *a, *b);
10 }
11
12 int main(int argc, char const *argv[])
13 {
14     int m = 5;
15     int n = 10;
16     swap(&m, &n);
17     printf("m=%d, n=%d\n", m, n);
18     return 0;
19 }
```

案例：指针/test4.c

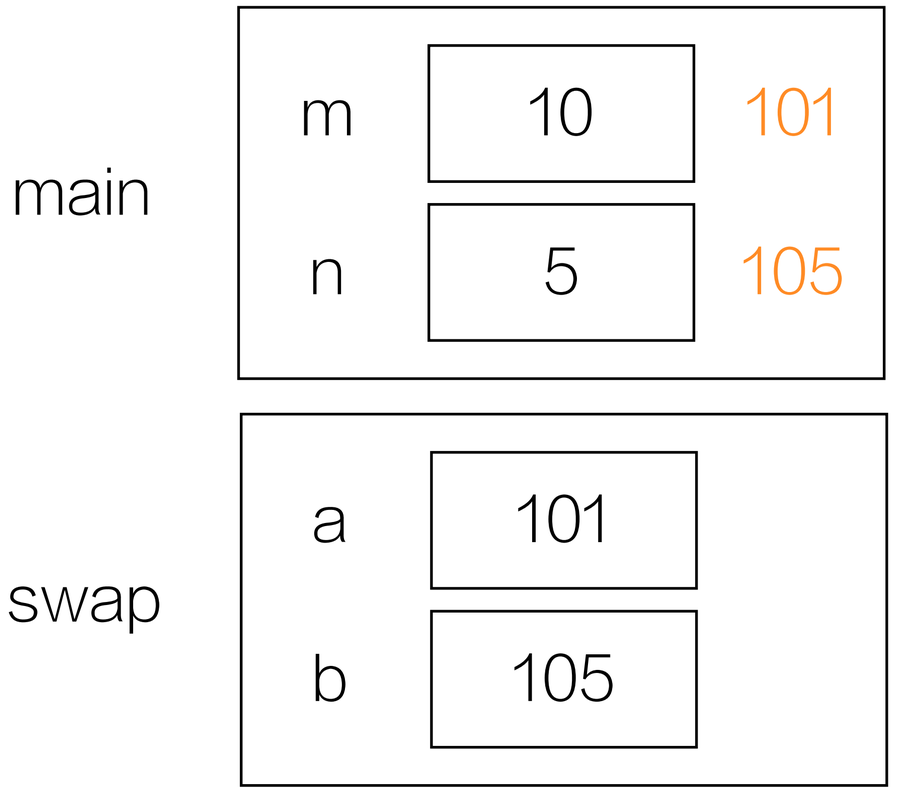


指针与函数

设计一个函数，传入两个 int 参数，并交换这两个参数的值。

```
3  void swap(int *a, int *b)
4  {
5      int temp;
6      temp = *a;
7      *a = *b;
8      *b = temp;
9      printf("a=%d, b=%d\n", *a, *b);
10 }
11
12 int main(int argc, char const *argv[])
13 {
14     int m = 5;
15     int n = 10;
16     swap(&m, &n);
17     printf("m=%d, n=%d\n", m, n);
18     return 0;
19 }
```

案例：指针/test4.c



结构体的声明

结构体是一个或多个变量的集合，这些变量可以是不同的类型。

声名语法：

```
struct 结构体名
{
    数据类型 变量名;
    数据类型 变量名;
    ……
};
```

```
struct point
{
    int x;
    int y;
};
```

案例：结构体/test1.c

结构体的初始化与调用

```
struct 结构体名 变量名;
```

```
struct point p;
```

```
变量名.结构体内部变量名 = 值
```

```
p.x = 10;
p.y = 15
```

案例：结构体/test1.c

结构体实例 – 创建一个 point

编写一个函数，传入两个int参数，在函数创建一个结构体point类型的变量，
将传入的两个参数分别赋值给该结构体变量的x和y，最后将该结构体变量返回。

```
struct point createPoint(int x, int y)
{
    struct point temp;
    temp.x = x;
    temp.y = y;
    return temp;
}
```

案例：结构体/test2.c

结构体与指针

在一些场景中，如果传递给函数的结构体很大，使用指针方式的效率通常更高。

pp指向一个point结构体

```
struct point *pp;
```

为pp所指向的结构体中的属性赋值

```
(*pp).x = 10;
(*pp).y = 5;
```

为了使用方便，C 语言提供了另外一种简写方式

```
pp->x = 10;
pp->y = 5;
```

案例：结构体/test3.c

类型定义

```
typedef 数据类型 别名
```

```
typedef int zx
```

案例：结构体/test4.c

类型定义

```c
struct point
{
    int x;
    int y;
};

int main(int argc, char const *argv[])
{
    struct point p;
    p.x = 5;
    p.y = 10;
    printf("%d\n", p.x);
    printf("%d\n", p.y);
    return 0;
}
```

每次声明结构体变量都要写 struct 关键字，很麻烦，而且逻辑上也很难受，typedef 可以解决这个问题。

类型定义

```
typedef struct 结构体名
{
    数据类型  变量名;
    数据类型  变量名;
    ……
}别名;
```

```
typedef struct
{
    数据类型  变量名;
    数据类型  变量名;
    ……
}别名;
```

类型定义

```
typedef struct point
{
    int x;
    int y;
}po;
```

```
typedef struct
{
    int x;
    int y;
}po;
```

案例：结构体/test5.c

内存分类

C 程序编译后，会以三种形式使用内存：

静态 / 全局内存

静态声明的变量和全局变量使用这部分内存，这些变量在程序开始运行时分配，直到程序终才消失。

自动内存（栈内存）

函数内部声明的变量使用这部分内存，在函数被调用时才创建。

动态内存（堆内存）

根据需求编写代码动态分配内存，可以编写代码释放，内存中的内容直到释放才消失。

动态内存分配

在C语言中，动态分配内存的基本步骤：

1. 使用 malloc（ memory allocate ）函数分配内存

void* malloc(size_t)

如果成功，会返回从堆内存上分配的内存指针

如果失败，会返回空指针

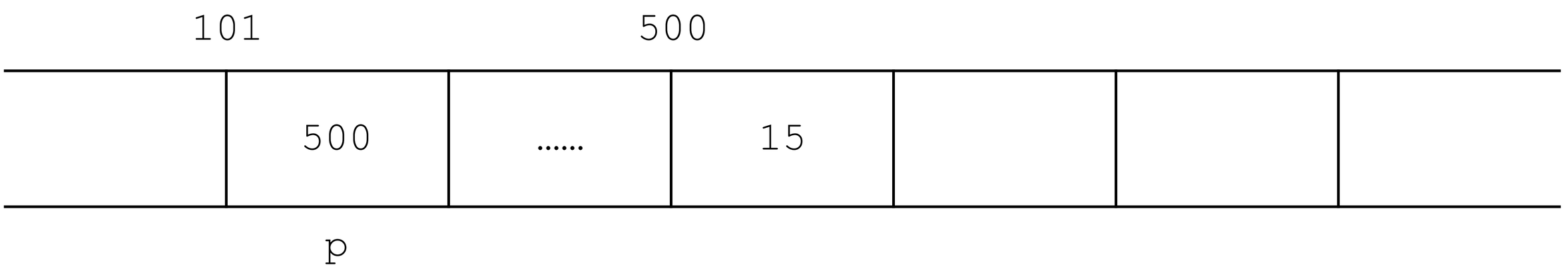
2. 使用分配的内存

3. 使用 free 函数释放内存

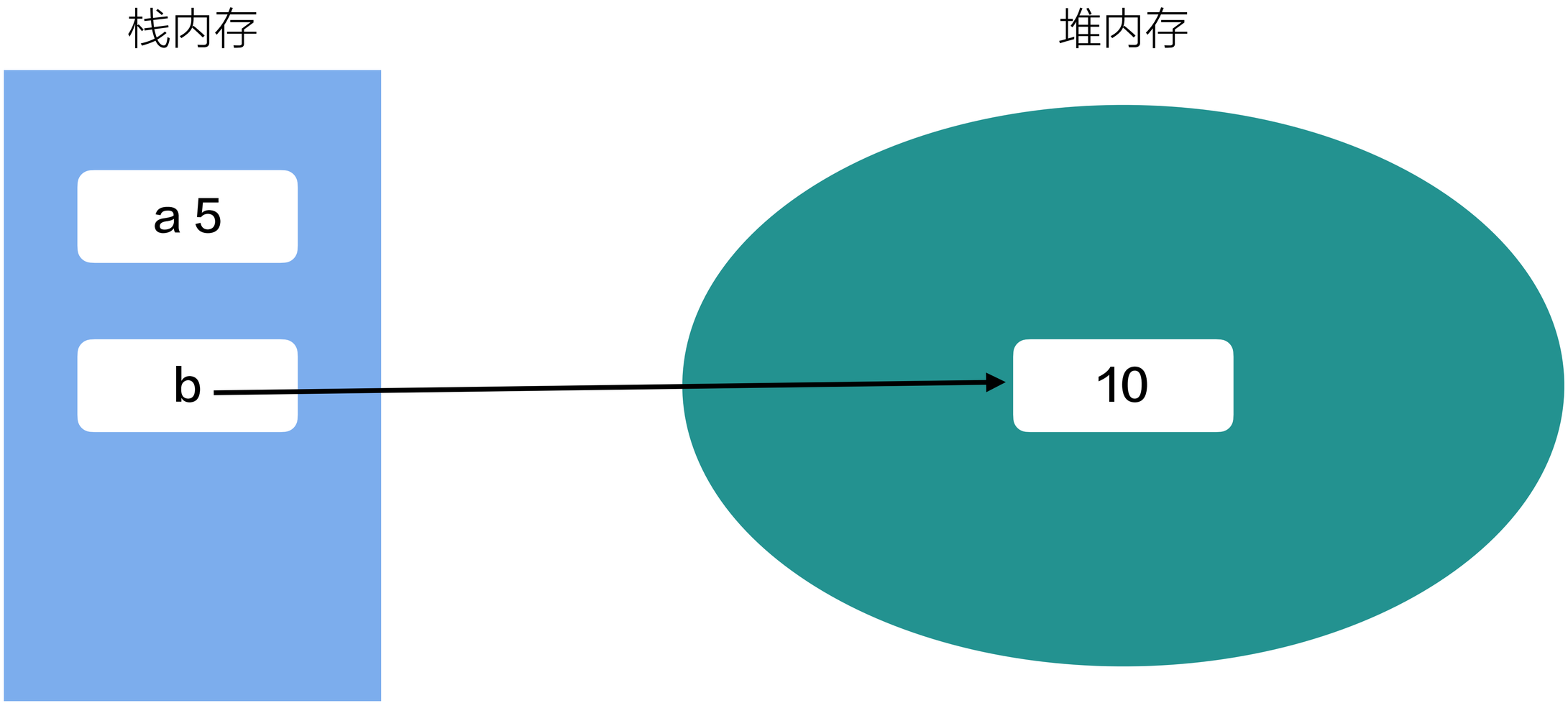
动态内存分配示例 – 基本数据类型

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char const *argv[])
{
    int *p;
    p = (int*)malloc(sizeof(int));
    *p = 15;
    printf("%d\n", *p);
    free(p);
    return 0;
}
```

案例：动态内存分配/test1.c



```c
int a = 5;
int *b = (int*)malloc(sizeof(int));
*b = 10;
```



动态内存分配示例 – 字符串

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char const *argv[])
{
    char *s;
    s = (char*)malloc(10);
    strcpy(s, "Hello");
    printf("%s\n", s);
    return 0;
}
```

案例：动态内存分配/test2.c

动态内存分配示例 – 数组

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char const *argv[])
{
    int *arr = (int*)malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++)
    {
        arr[i] = 0;
    }

    for (int i = 0; i < 5; i++)
    {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

案例：动态内存分配/test3.c

动态内存分配示例 – 结构体

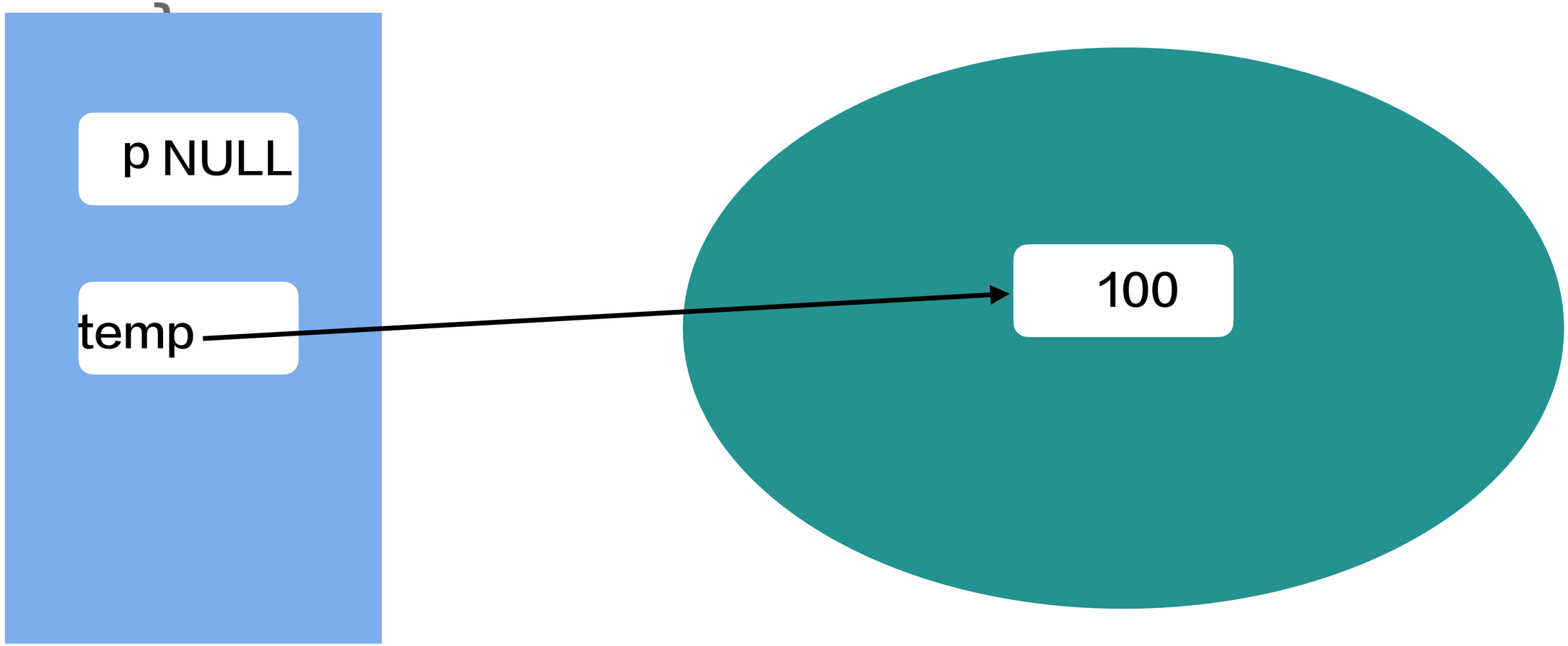
```c
#include <stdio.h>
#include <stdlib.h>
typedef struct
{
    int x;
    int y;
}po;

int main(int argc, char const *argv[])
{
    po *p;
    p = (po*)malloc(sizeof(po));
    p->x = 5;
    p->y = 10;
    printf("%d\n", p->x);
    printf("%d\n", p->y);
    return 0;
}
```

案例：动态内存分配/test4.c

二级指针

案例：动态内存分配/test5.c 案例：动态内存分配/test6.c

```
{
    int *p = NULL;
    fun(p);
    printf("%d\n", *p);
    return 0;
```



```
d fun(int** temp)

    *temp = (int*)malloc(sizeof(int));
    **temp = 100; //解引用两层指针，对上-
```